

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)**

**РАСШИРЕННЫЙ КОНСПЕКТ ДЛЯ ПРЕПОДАВАТЕЛЯ · ЛЕКЦИЯ
№ 4**

**СВЯЗНЫЕ СПИСКИ. СТЕК, ОЧЕРЕДЬ И ДЕК:
ПРЕДСТАВЛЕНИЕ, ИНВАРИАНТЫ И АБСТРАКТНЫЕ
ТИПЫ ДАННЫХ**

по дисциплине «Алгоритмы и структуры данных»

Направление подготовки: 03.03.01 Прикладные математика и физика

Направленность (профиль): 03.03.01 Прикладные физика и математика

Семестр: 7 · Объем лекции: 2 академических часа

Компетенция: ПК-4 · Индикатор: ПК-4.4

Ростов-на-Дону
2026

1. Паспорт лекции

Позиция	Содержание
Место в РПД	Раздел 2. Линейные структуры данных и абстрактные типы данных
Цель	Сформировать понимание связной организации данных и научить проектировать абстрактные типы Stack, Queue и Deque поверх связного списка и массива
Планируемый результат	Обучающийся объясняет инварианты связного списка, оценивает стоимость операций и обоснованно выбирает реализацию абстрактного типа данных под задачу
Связь с ПК-4.4	Использует современные прикладные и программные средства при решении задач профессиональной деятельности
Предварительная подготовка	Лекции № 1–3: спецификация и инварианты, асимптотический анализ, динамический массив и амортизированная стоимость
Текущий контроль	Устный разбор операций, трассировка вставки и удаления, разбор реализации стека, очереди и дека

2. Планируемые результаты по уровням

Уровень	После лекции обучающийся демонстрирует
Базовый	Различает односвязный и двусвязный список, узел и указатель; реализует базовые операции стека и очереди по спецификации
Средний	Оценивает стоимость операций списка, стека, очереди и дека; обоснованно выбирает реализацию (связный список, массив, кольцевой буфер) для типовой задачи
Продвинутый	Доказывает инварианты и корректность операций, включая разворот списка; проектирует дек для скользящего окна и объясняет ограничения выбранной реализации

3. Рекомендуемая организация 90 минут

Этап	Время	Деятельность
Проблемная постановка	0–8 мин	Почему известная позиция вставки не спасает массив, но спасает связную структуру
Узел и односвязный список	8–26 мин	Представление, инварианты, базовые операции и их стоимость
Двусвязный список	26–38 мин	Указатель prev, симметричные операции, удаление по известному узлу за $\Theta(1)$
Стек и очередь	38–58 мин	Контракты LIFO и FIFO, реализация поверх списка, роль хвостового указателя
Дек и его реализации	58–74 мин	Связный список против кольцевого буфера, локальность и накладные расходы
Мини-кейс и эксперимент	74–86 мин	Скользящий максимум на деке; измерение стоимости вставки в начало
Закрепление	86–90 мин	Выходной вопрос и постановка домашнего задания

4. Понятийный аппарат

Понятие	Рабочее определение
---------	---------------------

Узел (node)	Единица хранения связной структуры: поле данных и один или несколько указателей на соседние узлы
Связный список	Последовательность узлов, связанных указателями; логический порядок не требует физической смежности в памяти
Голова / хвост (head / tail)	Указатели на первый и, если поддерживается, последний узел списка
Указатель next / prev	Ссылка на следующий (next) или предыдущий (prev) узел; отсутствие связи обозначается специальным значением $\emptyset$ (null)
Абстрактный тип данных (ADT)	Интерфейс операций с контрактом, независимый от конкретной реализации структуры
Стек (LIFO)	ADT с операциями push и pop, обслуживающими только один открытый конец — вершину
Очередь (FIFO)	ADT с операциями enqueue и dequeue, различающими конец приёма и конец выдачи
Дек (deque)	ADT, допускающий вставку и удаление с обоих концов последовательности
Кольцевой буфер	Реализация очереди или дека поверх массива фиксированной емкости с циклической индексацией

5. От известной позиции к связной структуре

В лекции № 3 контракт динамического массива показал: вставка или удаление в позиции i стоит $\Theta(n-i)$, поскольку массив хранит элементы в смежных ячейках и обязан сохранять смежность после операции — любой элемент правее позиции i физически сдвигается. Даже если позиция известна заранее, стоимость сдвига не устраняется: она определяется числом элементов справа, а не сложностью поиска места.

Связная структура разрывает эту зависимость. Значение хранится в отдельном узле, а порядок задаётся не смежностью ячеек, а указателями. Если уже есть ссылка на нужный узел, вставка или удаление рядом с ним требует изменения фиксированного числа указателей — $\Theta(1)$ — независимо от длины списка. Плата за это — потеря прямого доступа по индексу: чтобы получить ссылку на i -й узел, нужно пройти список от головы, то есть $\Theta(i)$.

Не смешивать. $\Theta(1)$ для вставки по известному узлу не означает $\Theta(1)$ для вставки по известному номеру позиции — поиск узла по индексу в связном списке всё равно стоит $\Theta(i)$.

6. Односвязный список: представление и инварианты

6.1. Узел и структура списка

Узел односвязного списка хранит пару (value, next): значение и ссылку на следующий узел. Список задаётся указателем head на первый узел; последний узел имеет next = $\emptyset$. Пустой список — это head = $\emptyset$.

head $\rightarrow$ [5 | •] $\rightarrow$ [2 | •] $\rightarrow$ [9 | $\emptyset$]

6.2. Инвариант списка и контракт операции

Инвариант I: для непустого списка существует конечная последовательность узлов $n_0, \dots, n_{k-1}$, такая что head = n_0 , next(n_{k-1}) = $\emptyset$ (циклов нет), и для любого $0 \leq i < k-1$ выполняется

$\text{next}(n_i) = n_{i+1}$. Для пустого списка $\text{head} = \emptyset$. Инвариант гарантирует: список конечен, каждый узел достижим ровно одним путём от head , и обход по next завершается.

Контракт INSERT_FRONT(L, x). Предусловие — список L удовлетворяет инварианту I. Постусловие — создан новый узел v со значением x и $\text{next}(v) = \text{head_старый}$; $\text{head}(L) = v$; длина списка увеличилась на 1; относительный порядок прежних элементов не изменился; инвариант I сохранён.

Псевдокод

```
INSERT_FRONT(L, x)
```

```
  v ← new Node(x)
```

```
  v.next ← L.head
```

```
  L.head ← v
```

```
DELETE_FRONT(L)
```

```
  // предусловие: L.head ≠ ∅
```

```
  v ← L.head
```

```
  L.head ← v.next
```

```
  return v.value
```

7. Базовые операции односвязного списка и их стоимость

Операция	Худшая стоимость	Причина
INSERT_FRONT / DELETE_FRONT	$\Theta(1)$	Создание узла и переключение head не зависят от длины списка
INSERT_AFTER(v) / DELETE_AFTER(v)	$\Theta(1)$	Меняются указатели двух соседних звеньев по уже известной ссылке v
Поиск по значению / доступ по индексу i	$\Theta(n)$	Адресной формулы нет — обход от head обязателен
INSERT_BACK без хвостового указателя	$\Theta(n)$	Нужно дойти до последнего узла, чтобы присоединить новый
INSERT_BACK с хвостовым указателем tail	$\Theta(1)$	Прямой доступ к последнему узлу устраняет обход
DELETE_BACK (односвязный, без prev)	$\Theta(n)$	Нужно найти предпоследний узел, чтобы обнулить его next

Главный вывод. В связном списке дёшева работа рядом с уже известным местом — head, tail или конкретным узлом. Дорогим остаётся поиск места по значению или по номеру позиции.

8. Двусвязный список: зачем нужен prev

Узел двусвязного списка хранит тройку (prev, value, next). Список поддерживает head и tail; head.prev = $\emptyset$ и tail.next = $\emptyset$. Симметрия prev/next устраняет обход в двух ситуациях, где односвязный список требует $\Theta(n)$: удаление последнего узла (tail.prev доступен напрямую) и удаление произвольного узла по известной ссылке без поиска предшественника.

Контракт DELETE(L, v). Предусловие — v является узлом списка L. Постусловие — v.prev.next = v.next (или head(L) = v.next, если v = head); v.next.prev = v.prev (или tail(L) = v.prev, если v = tail); узел v исключён; порядок остальных узлов не изменился; инвариант списка сохранён.

Плата за симметрию — инженерная, а не асимптотическая: каждый узел хранит дополнительный указатель (рост памяти на элемент), а каждая операция связывания обновляет не одну, а две пары ссылок, что увеличивает число мест, где легко допустить ошибку в порядке присваиваний.

9. Стек (LIFO): контракт и реализация

Стек — абстрактный тип данных с одним открытым концом (вершиной), обслуживающим два наблюдаемых действия: добавление и извлечение самого последнего добавленного элемента.

Контракт стека. PUSH(S, x): постусловие — новый верхний элемент top(S) = x, порядок элементов под ним не изменился. POP(S): предусловие — S не пуст; постусловие — верхний элемент удалён и возвращён, порядок остальных не изменился.

Реализация поверх односвязного списка отождествляет top с head: PUSH — это INSERT_FRONT, POP — это DELETE_FRONT с возвратом значения. Обе операции используют единственный известный конец списка, поэтому обе стоят $\Theta(1)$ в худшем случае — обход никогда не требуется.

PUSH(S, x) $\equiv$ INSERT_FRONT(S, x)

POP(S) $\equiv$ DELETE_FRONT(S)

Прикладной пример: стек кадров рекурсивного вызова и стек отмены (undo) последних шагов итерационного численного метода — если очередное приближение расходится, метод откатывается к состоянию, снятому с вершины стека сохранённых шагов.

10. Очередь (FIFO): контракт и реализация

Контракт очереди. ENQUEUE(Q, x): постусловие — x добавлен в конец очереди, порядок остальных элементов не изменился. DEQUEUE(Q): предусловие — Q не пуста; постусловие — удалён и возвращён элемент из начала очереди, порядок остальных не изменился.

Очередь различает два конца: DEQUEUE удаляет из головы — это DELETE_FRONT, $\Theta(1)$. Но ENQUEUE добавляет в конец — без хвостового указателя это INSERT_BACK без tail, то есть $\Theta(n)$. Отсюда практическое требование: очередь на связном списке обязана хранить tail, иначе одна из двух операций гарантированно теряет $\Theta(1)$.

```
ENQUEUE(Q, x)

    v ← new Node(x)

    if Q.head = ∅

        Q.head ← v; Q.tail ← v

    else

        Q.tail.next ← v; Q.tail ← v

DEQUEUE(Q)

    // предусловие: Q.head ≠ ∅

    v ← Q.head

    Q.head ← v.next

    if Q.head = ∅

        Q.tail ← ∅

    return v.value
```

Связь с лекцией 3. Очередь на динамическом массиве без циклической индексации сталкивается с той же проблемой с другой стороны: DEQUEUE из начала массива требует сдвига всех оставшихся элементов, то есть $\Theta(n)$. Устранение этого сдвига — прямая причина, по которой очереди на массиве реализуют как кольцевой буфер (раздел 11).

11. Дек: контракт и две реализации

Дек (deque, double-ended queue) обобщает стек и очередь: PUSH_FRONT, PUSH_BACK, POP_FRONT и POP_BACK должны выполняться за $\Theta(1)$ в худшем случае на обоих концах.

11.1. Дек на двусвязном списке

Head и tail дают прямой доступ к обоим концам, а поле prev делает POP_BACK симметричным DELETE_FRONT: не нужен поиск предпоследнего узла. Все четыре операции — $\Theta(1)$ без исключений.

11.2. Дек на кольцевом буфере

Кольцевой буфер хранит элементы в непрерывном массиве емкости C с индексами head и tail по модулю C . Вставка с любого конца сдвигает соответствующий индекс на $1 \pmod C$ без переноса элементов. Когда буфер заполнен ($n = C$), требуется расширение — новый буфер большей емкости и копирование элементов в порядке от head до tail, ровно как геометрическое расширение

динамического массива из лекции 3. Амортизированная стоимость $PUSH_*$ остаётся $\Theta(1)$ по той же схеме удвоения.

Критерий	Дек на двусвязном списке	Дек на кольцевом буфере
$PUSH/POP$ с обоих концов	$\Theta(1)$ в худшем случае	$\Theta(1)$ амортизированно (расширение — $\Theta(n)$, но редко)
Память на элемент	Данные + два указателя на узел	Только данные; емкость может быть частично не занята
Локальность при обходе	Низкая: узлы разбросаны по памяти	Высокая: элементы физически смежны
Доступ по индексу	$\Theta(n)$ — только через обход	$\Theta(1)$ — прямое вычисление адреса с учётом модуля C

Терминологическая осторожность. Python `collections.deque` реализован как двусвязный список блоков фиксированного размера — не единичных элементов, что снижает накладные расходы на указатели. C++ `std::deque` хранит карту (map) указателей на отдельные фрагменты массива фиксированного размера. Обе реализации дают $\Theta(1)$ амортизированно на обоих концах, но не обязаны быть ни классическим связным списком, ни единым кольцевым буфером — важен контракт, а не конкретное внутреннее устройство.

12. Стоимость операций: связный список против массива

Операция	Динамический массив	Односвязный список	Двусвязный список
Доступ по индексу	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Вставка / удаление в начало	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Вставка в конец (амортизированно)	$\Theta(1)$	$\Theta(1)$ с tail / $\Theta(n)$ без tail	$\Theta(1)$
Удаление с конца	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$
Вставка / удаление по известной ссылке	$\Theta(n-i)$	$\Theta(1)$	$\Theta(1)$
Поиск по значению	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Память на элемент	Только данные (с запасом до C)	Данные + 1 указатель	Данные + 2 указателя
Локальность	Высокая	Низкая	Низкая

13. Разворот списка: алгоритм и доказательство корректности

`REVERSE(L)` переворачивает список за один проход, без дополнительной памяти на узлы, используя три указателя-курсора.

Псевдокод

```
REVERSE(L)
```

```
prev ← ∅
```

```
curr ← L.head
```

```
while curr ≠ ∅
```

```
    next ← curr.next
```

```
    curr.next ← prev
```

```
    prev ← curr
```

```
    curr ← next
```

```
L.head ← prev
```

Инвариант цикла $I(\text{prev}, \text{curr})$: множество узлов исходного списка разбито ровно на две непересекающиеся части — цепочку, начинающуюся в prev , в которой связи next уже развёрнуты в обратном порядке, и цепочку, начинающуюся в curr , в которой связи next совпадают с исходными и ни один узел ещё не тронут. Каждый узел принадлежит ровно одной из частей.

- Инициализация. До первой итерации $\text{prev} = \emptyset$ (развёрнутая часть пуста — условие выполнено вакуумно), а $\text{curr} = \text{L.head}$, то есть вся исходная последовательность целиком принадлежит нетронутой части. Инвариант выполнен.
- Сохранение. Пусть инвариант выполнен перед шагом и $\text{curr} \neq \emptyset$. Сохранение $\text{next} = \text{curr.next}$ фиксирует остаток нетронутой части до того, как curr.next будет перезаписан. Присваивание $\text{curr.next} \leftarrow \text{prev}$ присоединяет curr к развёрнутой части с правильным направлением связи. После $\text{prev} \leftarrow \text{curr}$ и $\text{curr} \leftarrow \text{next}$ развёрнутая часть увеличилась ровно на один узел (curr), нетронутая часть уменьшилась ровно на тот же узел; ни один узел не терялся и не дублировался, поскольку next был сохранён заранее. Инвариант выполнен для новых значений prev и curr .
- Завершение. Цикл останавливается, когда $\text{curr} = \emptyset$, то есть нетронутая часть пуста, а развёрнутая часть содержит все n узлов исходного списка с полностью обращённым порядком связей. Присваивание $\text{L.head} \leftarrow \text{prev}$ делает первым узлом бывший последний узел исходного списка. Каждая итерация выполняет постоянное число операций, всего n итераций, поэтому REVERSE работает за $\Theta(n)$ при $O(1)$ дополнительной памяти.

Граница доказательства. Приведённое рассуждение доказывает частичную корректность (если алгоритм завершится, список развёрнут верно) и отдельно — завершение (curr монотонно проходит конечное число узлов к $\emptyset$, если исходный список не содержит цикла, что гарантировано инвариантом односвязного списка из раздела 6).

14. Мини-кейс: скользящий максимум на деке

Пусть $x[0..N-1]$ — временной ряд отсчётов датчика физического эксперимента. Нужно для каждого окна длины w (окно $[i-w+1, i]$) вычислить максимум за суммарное время $\Theta(N)$, а не $\Theta(N \cdot w)$, которое даёт наивный перебор каждого окна заново.

Решение хранит в деке индексы отсчётов, а не сами значения, поддерживая инвариант: индексы в деке упорядочены по возрастанию времени и одновременно по невозрастанию значений x . Голова дека всегда указывает на индекс текущего максимума окна.

Псевдокод

```
SLIDING_MAX(x, N, w)

D ← empty_deque() // хранит индексы

result ← []

for i ← 0 to N - 1

    while D not empty and x[front(D)] out of window(i, w)

        POP_FRONT(D)

    while D not empty and x[back(D)] ≤ x[i]

        POP_BACK(D)

    PUSH_BACK(D, i)

    if i ≥ w - 1

        result.append(x[front(D)])

return result
```

Каждый индекс попадает в дек через `PUSH_BACK` ровно один раз и может быть удалён не более одного раза — либо через `POP_FRONT` при выходе из окна, либо через `POP_BACK` при вытеснении бóльшим значением. Суммарное число операций дека не превышает $2N$, поэтому суммарная стоимость $\Theta(N)$, а амортизированная стоимость шага цикла — $\Theta(1)$. Это тот же аргумент, что и в амортизированном анализе динамического массива из лекции 3: дорогие внутренние удаления оплачиваются заранее выполненными вставками того же элемента.

Связь с ПК-4.4. Задание студенту — реализовать `SLIDING_MAX` на выбранном языке, измерить время на реальном временном ряде эксперимента для нескольких w и N , сравнить с наивным $\Theta(N \cdot w)$ вариантом и явно указать ограничение выбранной реализации: память дека растёт с длиной окна w , а не с N .

15. Экспериментальная проверка

Гипотеза: серия из N вставок в начало последовательности растёт линейно по времени для связного списка (каждая вставка $\Theta(1)$) и квадратично для динамического массива (каждая вставка $\Theta(n)$, суммарно $\Theta(N^2)$).

```

for N in 2^8, ..., 2^18:

    L ← empty_linked_list()

    A ← empty_dynamic_array()

    t1 ← measure( repeat N times: INSERT_FRONT(L, value) )

    t2 ← measure( repeat N times: A.insert(0, value) )

    save N, t1, t2

```

При интерпретации результата нужно отделять аллокацию узла (управляется менеджером памяти или сборщиком мусора) от собственно переключения указателей, и учитывать, что накладные расходы на мелкие объекты-узлы в языках с автоматическим управлением памятью могут маскировать асимптотический выигрыш на умеренных N .

16. Вопросы для взаимодействия

- Список содержит 10 узлов, известна ссылка на 5-й узел. Сколько указателей меняется при вставке нового узла сразу после него в односвязном и в двусвязном списке?
- Почему очередь на односвязном списке без хвостового указателя не может дать $\Theta(1)$ одновременно для enqueue и dequeue?
- Дек на кольцевом буфере заполнен ($n = C$). Что произойдёт при PUSH_BACK и какова стоимость этой операции в худшем и амортизированном смысле?
- Какие данные эксперимента естественно обрабатывать стеком, а какие — очередью, и почему?
- Что изменится в инварианте скользящего максимума, если в окне встречаются равные значения x ?

17. Типичные ошибки

- терять единственную ссылку на голову при обходе списка — перезаписывать head вместо временной переменной-курсора;
- вставлять или удалять узел, не сохранив заранее нужную ссылку (next или prev), и тем самым терять доступ к остатку списка;
- забывать обновлять tail при удалении последнего узла — очередь остаётся с висящим указателем на несуществующий элемент;
- путать порядок присваиваний указателей при развороте или вставке, из-за чего возникает цикл или теряется часть списка;
- считать реализацию через связный список автоматически быстрее массива — при последовательном обходе локальность массива часто перевешивает выигрыш $\Theta(1)$ на отдельной операции;
- не проверять предусловие POP/DEQUEUE на пустой структуре;
- использовать дек на кольцевом буфере без обработки переполнения при $n = C$.

18. Итоги лекции

- Связная структура заменяет прямую адресацию указателями: вставка и удаление по известному узлу стоят $\Theta(1)$ ценой $\Theta(n)$ для доступа по индексу.
- Двусвязный список за счёт prev делает удаление последнего узла и удаление по известной ссылке $\Theta(1)$ без поиска предшественника.

- Стек и очередь — это контракты (LIFO и FIFO), а не структуры данных; их можно реализовать поверх связанного списка или массива, если выполнен контракт стоимости.
- Хвостовой указатель — обязательное условие $\Theta(1)$ для добавления в конец очереди на связанном списке.
- Дек допускает $\Theta(1)$ на обоих концах и реализуется либо двусвязным списком, либо кольцевым буфером с тем же амортизированным анализом, что и динамический массив.
- Корректность алгоритмов над указателями (например, разворота списка) доказывается тем же методом инварианта цикла, что и корректность алгоритмов над массивами.

19. Контрольные вопросы

1. Почему известная позиция вставки не устраняет стоимость $\Theta(n-i)$ для динамического массива, но устраняет её для связанного списка?
2. Сформулируйте инвариант односвязного списка и объясните, что он гарантирует.
3. Запишите контракт $\{P\} A \{Q\}$ для INSERT_FRONT и для DELETE_AFTER(v).
4. Почему двусвязный список делает DELETE_BACK операций $\Theta(1)$, а односвязный — нет?
5. Сформулируйте контракты стека и очереди. Чем они отличаются друг от друга?
6. Почему очередь на односвязном списке без хвостового указателя теряет $\Theta(1)$ хотя бы для одной из двух операций?
7. Какие операции обязаны быть $\Theta(1)$ в худшем случае для корректной реализации дека?
8. Сравните дек на двусвязном списке и дек на кольцевом буфере по локальности и по памяти на элемент.
9. Изложите три части доказательства корректности алгоритма REVERSE через инвариант цикла.
10. Почему амортизированный анализ дека на кольцевом буфере повторяет анализ динамического массива из лекции 3?
11. В чём состоит инвариант дека индексов в алгоритме скользящего максимума?
12. Почему суммарная стоимость алгоритма скользящего максимума равна $\Theta(N)$, а не $\Theta(N \cdot w)$?
13. Приведите пример типичной ошибки при вставке узла в связный список и объясните её следствие.
14. Какие данные потребуются, чтобы экспериментально подтвердить гипотезу $\Theta(1)$ против $\Theta(n)$ для вставки в начало последовательности?

20. Домашнее задание

- Реализовать односвязный список с операциями insert_front, insert_after, delete_after, delete_front, search и обходом, соблюдая контракты из раздела 6.
- На основе списка реализовать Stack (push/pop) и Queue (enqueue/dequeue с хвостовым указателем); написать тесты на пустой структуре и структуре из одного элемента.
- Реализовать SLIDING_MAX на деке для синтетического временного ряда, измерить время для нескольких N и w , сравнить с наивной $\Theta(N \cdot w)$ реализацией.
- Провести эксперимент из раздела 15: сравнить время серии вставок в начало для связанного списка и динамического массива, построить график зависимости от N и объяснить форму кривых.
- Подготовиться к лекции № 5: вспомнить, как оценивается число сравнений при сортировке массива, и предположить, меняется ли это число при сортировке связанного списка.

21. Основная и дополнительная литература

- Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 4th ed. MIT Press, 2022. Chapter 10. Elementary Data Structures.

- Morin P. Open Data Structures. Linked Lists. Открытый учебник: https://opendatastructures.org/ods-python/3_Linked_Lists.html (дата обращения: 07.09.2026).
- Python Software Foundation. Python 3.14 Documentation. collections.deque: <https://docs.python.org/3/library/collections.html#collections.deque> (дата обращения: 07.09.2026).
- cppreference.com. std::list, std::forward_list, std::deque: <https://en.cppreference.com/w/cpp/container> (дата обращения: 07.09.2026).
- Sedgewick R., Wayne K. Algorithms. 4th ed. Addison-Wesley, 2011. Section 1.3. Bags, Queues, and Stacks.
- Актуализированная РПД дисциплины «Алгоритмы и структуры данных» для направления 03.03.01, редакция от 06.09.2026.

Методические заметки к презентации

Раздел дополняет основной текст сценарием объяснения по слайдам. Формулировки предназначены для устного раскрытия материала, постановки вопросов аудитории и фиксации переходов между понятиями.

Слайд 1. АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ

Свяжите тему с лекцией 3: контракт динамического массива подвёл к вопросу — можно ли устранить стоимость сдвига, если позиция вставки уже известна.

Слайд 2. ПРОБЛЕМНЫЙ ВОПРОС

Соберите гипотезы студентов. В конце лекции вернитесь к вопросу и явно разделите $\Theta(1)$ по узлу и $\Theta(i)$ по индексу.

Слайд 3. После лекции вы сможете

Озвучьте проверяемые результаты: контракт операции, трассировка указателей, обоснованный выбор реализации ADT.

Слайд 4. Логика лекции: от указателя к инженерному выбору

Дайте студентам маршрут лекции целиком, прежде чем погружаться в детали односвязного списка.

Слайд 5. Связный список и массив — два способа хранить порядок

Подчеркните симметрию: массив платит за вставку в известную позицию, список платит за доступ по индексу.

Слайд 6. Состояние узла и списка задают несколько полей

Явно проговорите: узел — единица хранения, список — это состояние head (и tail), а не отдельный контейнер данных.

Слайд 7. СВЯЗНЫЙ СПИСОК

Контраст с лекцией 3: $\text{addr}(A[i]) = B + i \cdot s$ вычисляется, $\text{next}(v)$ — хранится явно в самом узле.

Слайд 8. Каждый узел ссылается на следующий по указателю

Проведите пальцем по цепочке на слайде: head, затем каждый next, до $\emptyset$ — это и есть обход списка.

Слайд 9. $\Theta(1)$ вставки по узлу не отменяет предусловий

Как и в лекции 3 с $\Theta(1)$ доступа по индексу: быстрая операция всё равно требует выполненного предусловия.

Слайд 10. Операции списка тоже имеют разную цену

Это разделение — центральный тезис лекции: дешево рядом с известным местом, дорого — искать место.

Слайд 11. Связный список теряет то, что давал массив, — локальность

Явная переключка с локальностью из лекции 3, но с обратным знаком: там она работала на массив, здесь — против списка.

Слайд 12. Известный узел не спасает от поиска по позиции

Прямой ответ на проблемный вопрос слайда 1 — зафиксируйте его словами студентов.

Слайд 13. Вставка после известного узла — четыре шага

Это самая частая ошибка раздела «типичные ошибки» — проиллюстрируйте её на доске в реальном времени.

Слайд 14. Стек и очередь — это контракт, а не структура

Подчеркните: LIFO/FIFO — это контракт интерфейса; связный список — лишь одна из возможных реализаций.

Слайд 15. Один интерфейс, два открытых конца — push/pop с обеих сторон

Дек — естественное обобщение: то, что стек и очередь используют по одному концу, дек использует оба.

Слайд 16. Дек на двусвязном списке даёт $\Theta(1)$ на обоих концах без исключений

Именно prev делает POP_BACK симметричным DELETE_FRONT — без него пришлось бы искать предпоследний узел.

Слайд 17. Дек на кольцевом буфере повторяет анализ динамического массива

Это тот же геометрический аргумент из лекции 3 — расширение дека на кольцевом буфере ничем не отличается.

Слайд 18. Дек можно реализовать разными способами с одним контрактом

Реальные библиотечные деки нередко устроены сложнее учебной модели (блоки, карты фрагментов) — контракт от этого не меняется.

Слайд 19. ИНЖЕНЕРНЫЙ ВЫБОР

Переход от доказанных гарантий к инженерному решению — как и в лекции 3 с коэффициентом роста.

Слайд 20. Компромисс присутствует и в реализации дека

Проведите параллель с малым/большим α из лекции 3 — здесь то же решение, но на уровне выбора структуры целиком.

Слайд 21. Одна абстракция ADT — разные реализации на практике

Терминологическая осторожность: ни один из двух реально не является ни «чистым» связным списком, ни единым кольцевым буфером.

Слайд 22. Скользящий максимум на deque — $\Theta(N)$, а не $\Theta(N \cdot w)$

Дек хранит индексы, а не значения; инвариант — невозрастание значений от головы к хвосту.

Слайд 23. Типичные ошибки со связными структурами

Каждая ошибка встречается в домашних работах ежегодно — стоит явно проговорить следствие каждой.

Слайд 24. Мини-задача: разворот короткого списка

Попросите студентов проговорить три части инварианта из раздела 13 конспекта на этом конкретном примере.

Слайд 25. Проверьте себя перед следующей лекцией

Выходной билет: пусть каждый студент письменно ответит на один вопрос по выбору перед уходом.

Слайд 26. ИТОГ

Замкните лекцию на вопрос со слайда 1 и явно анонсируйте тему лекции 5.

Источники, использованные в заметках

- Актуализированная РПД «Алгоритмы и структуры данных», 03.03.01, редакция от 06.09.2026.
- Morin P. Open Data Structures. Linked Lists. https://opendatastructures.org/ods-python/3_Linked_Lists.html
- Cormen T. H. et al. Introduction to Algorithms. 4th ed. MIT Press, 2022. Chapter 10.
- Sedgewick R., Wayne K. Algorithms. 4th ed. Addison-Wesley, 2011. Section 1.3.
- Hoare C. A. R. An Axiomatic Basis for Computer Programming. Communications of the ACM. 1969.
- Python Software Foundation. Python 3.14 Documentation. `collections.deque`.
- `cppreference.com`. `std::list`. <https://en.cppreference.com/w/cpp/container/list>
- Cormen T. H. et al. Introduction to Algorithms. 4th ed. MIT Press, 2022. Amortized analysis.
- `cppreference.com`. `std::deque`. <https://en.cppreference.com/w/cpp/container/deque>